integrates different authorization patterns—ownership baselines, customer view policies, discretionary sharing, and global constraints—into a unified framework. Along the way, we examined how the entity store and relationship graph serve as the data sources for policies, and how requests travel through the Policy Enforcement Point to Cedar for evaluation.

We also compared deployment methods, including running Cedar locally with the Cedar Local Agent and using Amazon Verified Permissions as a managed service. To ensure policies function as intended, we examined testing strategies and analysis tools that help verify decisions before deployment.

To see all the examples we've covered in this chapter connected in one flow, refer to the appendix for a complete end-to-end walkthrough of ACME's Customer Collaboration platform. There, you can follow along step by step —from defining the schema to running real is-authorized calls—and try the full scenario yourself.

Next, we'll step back from implementation details and focus on designing effective authorization policies: how to consider policy as code, avoid common pitfalls, and align policies with organizational needs.

9.10 Summary

- A Cedar schema provides the foundation for safe, consistent policies by defining entity types, attributes, actions, and context.
- Schema design choices—like naming conventions and typed relationships—directly shape policy clarity and maintainability.
- ACME's core policies set the baseline: owners can act on their own documents, readers teams can view, and managers can supervise.
- The entity store, modeled as a relationship graph, provides the dynamic data that drives Cedar's static policy logic.
- Cedar supports discretionary, membership, and relationship permissions, providing flexibility to combine RBAC, ABAC, and ReBAC patterns.